

Diabetic Retinopathy Stage Screening — Product Guide

Audience: product owners, engineers, and operators who need the full story of this system.

Not a coursework report — this is the production-oriented product bible.

Status: EXP-ACC-070 export (2026-10-01). Test split never used for training.

Deployed winner: CORAL · test Acc **0.651** · QWK **0.738** · threshold **0.525** · `models/best_model.keras`

Table of contents

1. Product overview
 2. Problem and clinical concepts
 3. Data acquisition
 4. Cleaning and dataset preparation
 5. Preprocessing pipeline
 6. Augmentation and class balancing
 7. Model concepts
 8. Training operations
 9. Results
 10. Demo and product UX
 11. Limitations
 12. Accuracy upgrade roadmap
 13. Glossary
 14. How to reproduce
-

1. Product overview

What this product is

An automated **screening assistant** that grades color fundus photographs into five International Clinical Diabetic Retinopathy (ICDR) stages:

Code	Stage	Meaning for triage
0	No_DR	No diabetic retinal lesions
1	Mild	Microaneurysms only
2	Moderate	More than mild; less than severe
3	Severe	Extensive hemorrhages / venous beading / IRMA
4	Proliferative_DR	Neovascularization or vitreous/preretinal hemorrhage

What it is not

- Not a medical device.
- Not a replacement for an ophthalmologist.
- Not approved for autonomous diagnosis or treatment decisions.

Outputs are **decision support**: stage label, confidence / risk framing, and Grad-CAM heatmaps so a human can see *where* the network attended.

Who uses it

Role	Use
Screening clinic staff	Upload fundus images; prioritize high-risk grades for specialist review
Engineers / ML ops	Retrain, evaluate, export <code>best_model.keras</code> + <code>meta.json</code>
Demo stakeholders	Streamlit UI with preprocess preview + Grad-CAM

What ships

```
models/best_model.keras  ← deployed network (CORAL winner)
models/meta.json         ← winner flag, class names, metrics, preprocess knobs
app/streamlit_app.py     ← real-time demo
src/dr_detect/           ← reusable package (data, preprocess, models, train, metrics, explain)
scripts/                 ← CLI: download, train, evaluate, data sanity, diagrams
```

End-to-end system architecture



2. Problem and clinical concepts

Why DR screening matters

Diabetic retinopathy damages retinal microvasculature. Untreated progression can cause irreversible vision loss. Specialist grading capacity is limited; automation helps **triage** — put Severe and Proliferative cases in front of clinicians sooner.

What the model must learn to “see”

- **Microaneurysms** — tiny outpouchings of capillaries (early Mild).
- **Hemorrhages / exudates / cotton-wool spots** — Moderate–Severe disease load.
- **Neovascularization / vitreous hemorrhage** — Proliferative DR (highest urgency).

Why grades are ordered

Mild → Moderate is clinically closer than Mild → Proliferative. A model that confuses Mild with Moderate is less dangerous than one that calls Proliferative “No_DR”. That is why this product optimizes and reports **Quadratic Weighted Kappa (QWK)** alongside accuracy:

- QWK rewards adjacent mistakes and punishes far mistakes.
- Accuracy alone can look “OK” while still making dangerous jumps across the scale.

Softmax vs ordinal thinking

- **Softmax** treats classes as unrelated buckets (classical multi-class).
- **CORAL** models cumulative thresholds: $P(y > 0)$, $P(y > 1)$, $P(y > 2)$, $P(y > 3)$ — naturally ordinal and aligned with QWK.

3. Data acquisition

Primary dataset

Field	Value
Source	Kaggle <code>sachinkumar413/diabetic-retinopathy-dataset</code>
Approximate size	~350 MB download
Images used	2750 labeled fundus images
Structure	Folders per severity

Folder → stage map

Folder name	Internal stage
Healthy	No_DR (0)
Mild DR	Mild (1)
Moderate DR	Moderate (2)
Severe DR	Severe (3)
Proliferate DR	Proliferative_DR (4)

Download is automated via the Kaggle API (`scripts/download_dataset.py` / `dr_detect.data.download_kaggle_dataset`), with credentials in a locked-down `kaggle.json` (`chmod 600`).

Ethics and data governance

- Public secondary research data — **no PHI** stored in the repo by design.
- Labels can be noisy (single-grader bias possible).
- No guaranteed patient-level IDs in this pack → risk of eye-pair leakage; document as a limitation.
- Outputs remain screening support only.

Why this dataset (vs larger EyePACS / APTOS packs)

Chosen for a complete multi-class download on constrained local hardware, five explicit folders, and room to own the full preprocess stack (CLAHE / Graham / unsharp) rather than relying on a pre-filtered pack.

4. Cleaning and dataset preparation

Indexing

1. Walk `data/raw/` and build a DataFrame of `image_path`, `label`, `label_name`.
2. Persist the full index as `data/processed/all_images.csv`.
3. Stratified split (SEED=42): 70% / 15% / 15% train / val / test.
4. Write `train.csv`, `val.csv`, `test.csv`, `split_stats.json`, `class_weights.json`.

Split counts (locked for all stable experiments)

Split	N
Train	1924
Val	413
Test	413
Total	2750

Train class counts: No_DR 699 · Mild 259 · Moderate 630 · Severe 133 · Proliferative_DR 203.

Severe is the scarcest class (~7% of train) — the main imbalance driver.

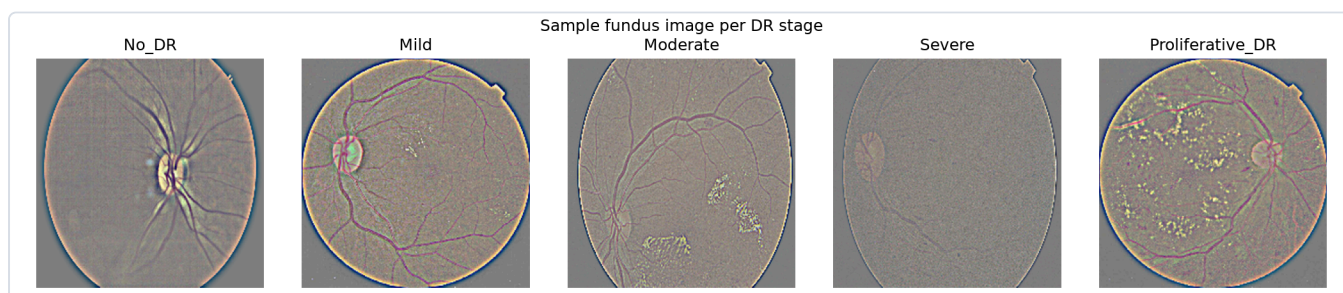
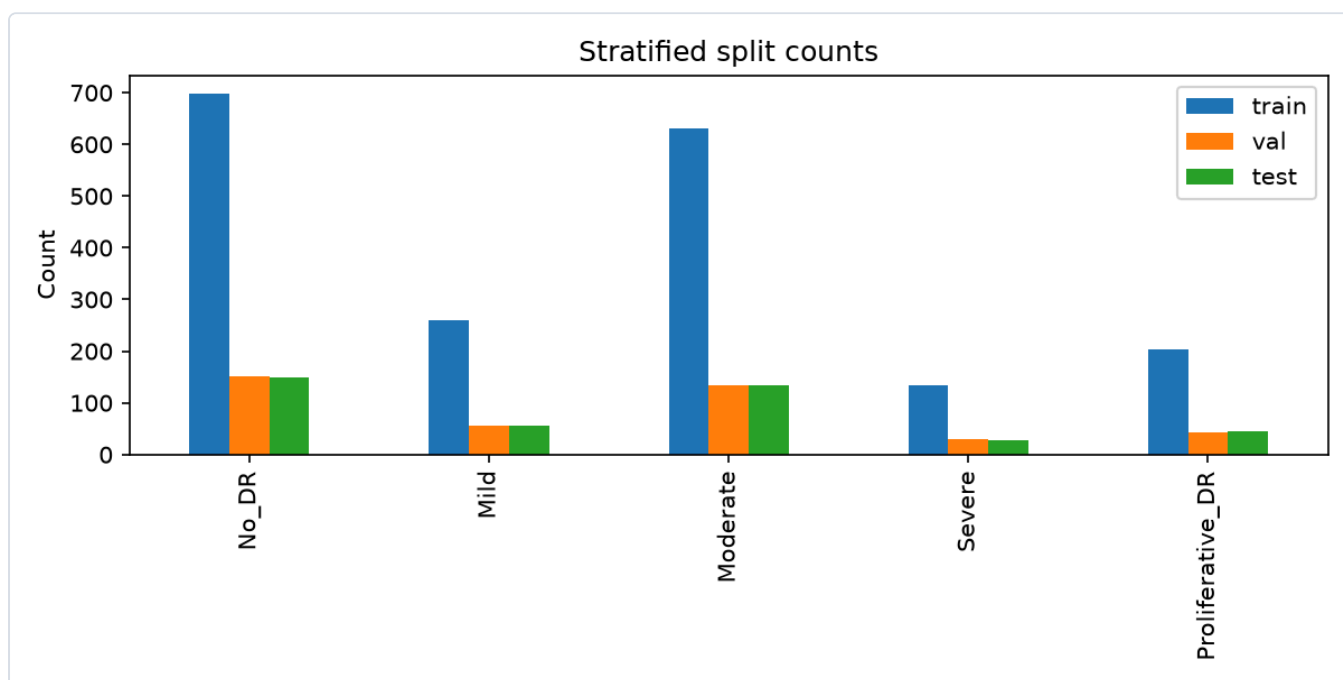
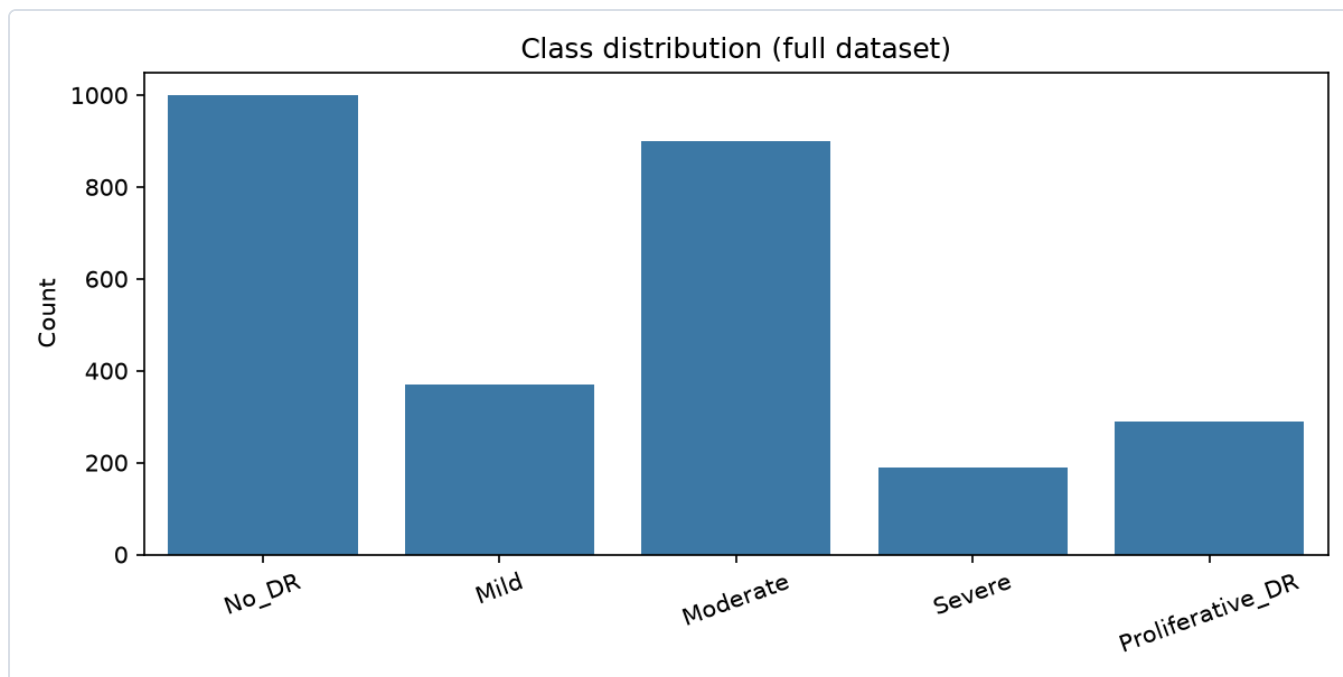
Extra training images (EXP-ACC-070)

APTOS 2019 at 224 px (`sovit Rath/diabetic-retinopathy-224x224-2019-data`) was added to the **training split only**. File-hash and 16×16 average-hash checks dropped any photo already in train, validation, or test.

	N
APTOS images found	3662
Duplicates dropped	1124
Extra train rows kept	2538
Pooled train before oversample	4462
Validation (unchanged)	413
Test (unchanged)	413

Extra rows by stage: No_DR 1183 · Mild 307 · Moderate 673 · Severe 147 · Proliferative_DR 228.

Pooled train before oversample: No_DR 1882 · Mild 566 · Moderate 1303 · Severe 280 · Proliferative_DR 431. Mild, Severe, and Proliferative were then duplicated up to 1882 rows each, so the fit table has **8831** rows. Class weights were not used on this run.



Data sanity check

`scripts/data_sanity.py` samples train images and records:

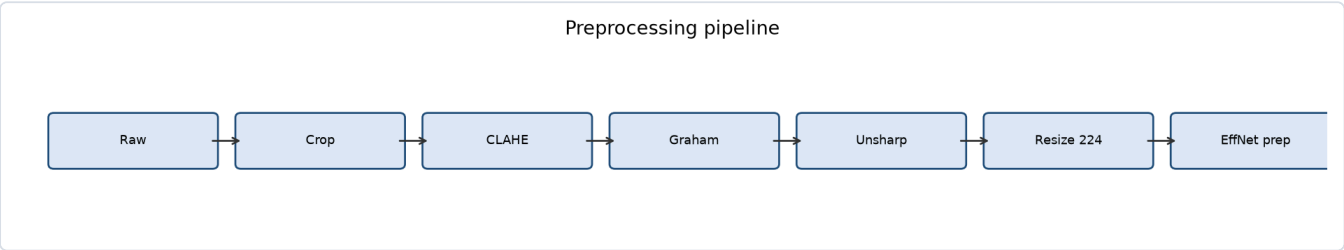
Check	Result (stable run)
Scanned	500
Unreadable	0
Near-black (mean < 5)	0

Artifact: `data/processed/data_sanity.json` .

Shuffle runs every epoch in `FundusSequence.on_epoch_end` so batches are not stuck in a fixed bad order.

5. Preprocessing pipeline

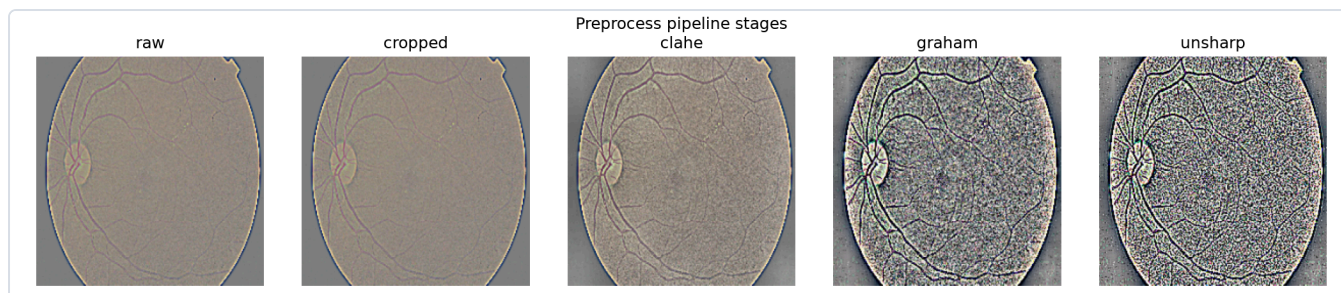
Raw camera frames vary in illumination, fringe darkness, and contrast. The product applies a fixed **hybrid fundus preprocess** before both training and inference so the model always sees the same distribution.



Steps (in order)

#	Step	Why
1	Dark-border crop (<code>tol=7</code>)	Remove black framing so the retina fills the frame
2	CLAHE on LAB L-channel (<code>clipLimit=2.0</code> , tiles 8×8)	Local contrast — microaneurysms / vessels stand out
3	Ben Graham illumination (<code>addWeighted</code> 4/-4 + Gaussian $\sigma=10$)	Classic DR competition trick for lighting normalization
4	Unsharp mask ($\sigma=1.0$, <code>amount=1.5</code>)	Edge enhancement for lesion boundaries
5	Resize 224×224	EfficientNet input size
6	EfficientNet <code>preprocess_input</code>	Match ImageNet transfer-learning scale

Implementation: `src/dr_detect/preprocess.py` · knobs in `src/dr_detect/config.py` .

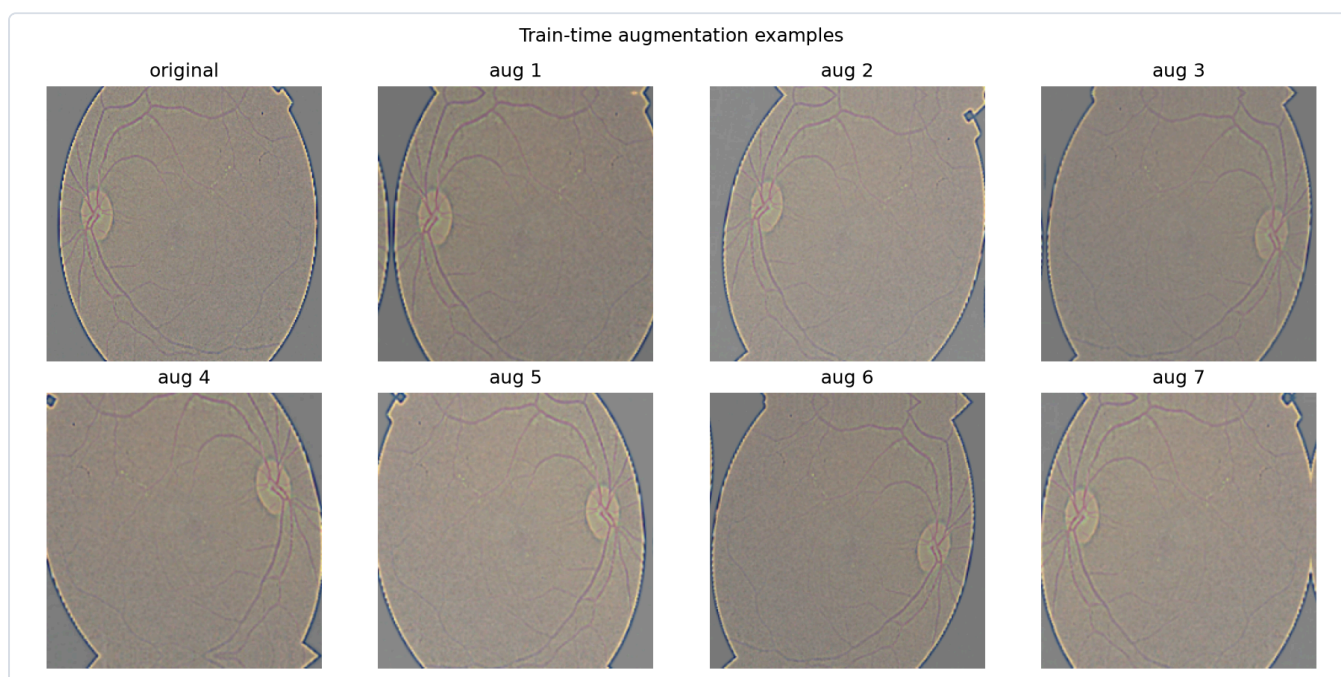


6. Augmentation and class balancing

Augmentation (train only)

Never augment val/test — that would leak optimistic metrics.

Typical transforms (see `src/dr_detect/augment.py`): horizontal flip, small rotation ($\sim \pm 15^\circ$), zoom, brightness/contrast jitter, small translation.



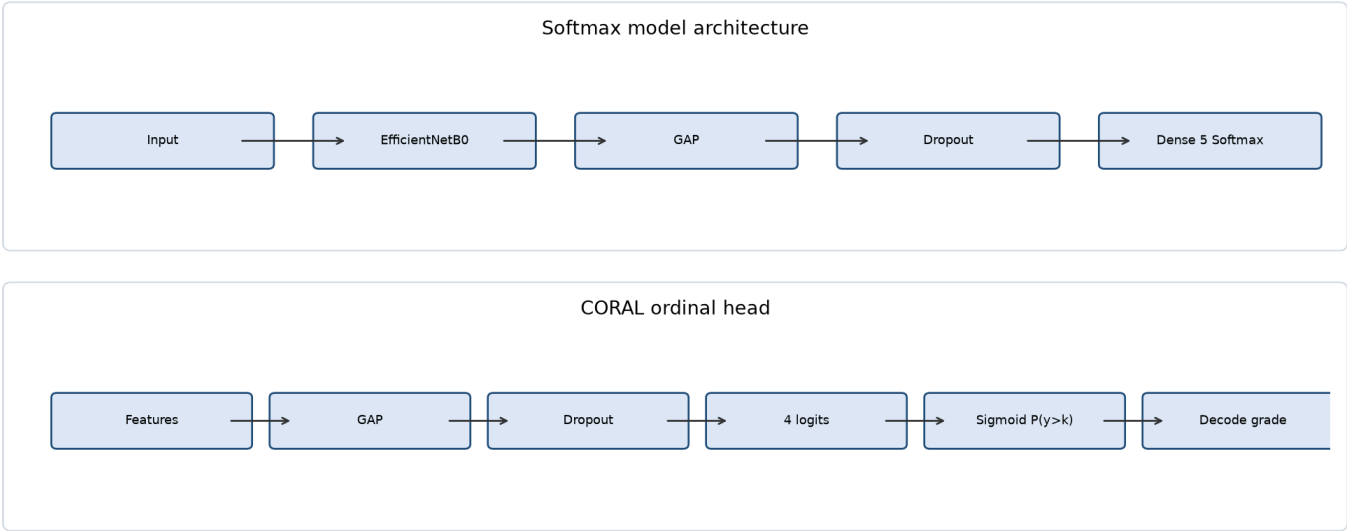
Class weights

`sklearn` balanced weights from the train split (stored in `data/processed/class_weights.json`):

Class	Weight (approx.)
0 No_DR	0.55
1 Mild	1.49
2 Moderate	0.61
3 Severe	2.89
4 Proliferative	1.90

Softmax training in the earlier scripts uses these weights. The deployed EXP-ACC-070 run did not. It balanced the pooled training rows by duplication only.

7. Model concepts



Backbone: EfficientNetB0

- ImageNet pretrained, `include_top=False`, input **224×224×3**.
- Chosen for local RAM (~7 GB) and CPU inference; B3 remains a GPU stretch goal (see roadmap).

Softmax head

```
EfficientNetB0 → GlobalAveragePooling2D → Dense(512) → Dropout(0.3) → Dense(256) → Dropout(0.3) → Dense(5, softmax)
```

The deployed EXP-ACC-070 run used this head with label smoothing off and no class weights.

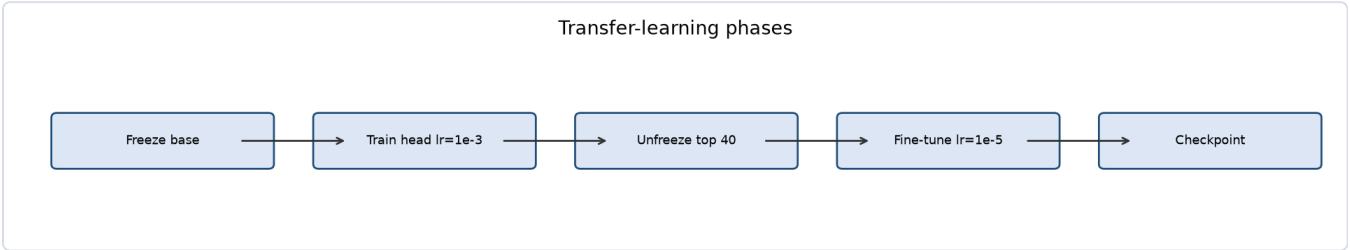
CORAL ordinal head

```
EfficientNetB0 → GAP → Dense(512) → Dropout(0.3) → Dense(256) → Dropout(0.3) → Dense(4, linear)
```

The four logits are cumulative thresholds. The deployed cutoff is 0.525, chosen on the validation set.

- Loss: binary cross-entropy on cumulative levels (CORAL loss).
- Decode: sigmoid → count how many thresholds fire → integer grade 0–4.
- **Do not** trust Keras “accuracy” on raw CORAL logits — that metric is misleading.

Transfer learning: two phases (BN-safe)



Phase	What trains	LR	Epochs (cap)
1	Head only (backbone frozen)	3e-4	up to 8
2	Last 20 non-BN backbone layers	1e-5	up to 8

Stability rules (critical product knowledge):

1. Keep all **BatchNormalization** layers frozen / inference-mode during fine-tune (Keras TL recipe — thawing BN destroys ImageNet running stats and caused earlier mid-run collapse).
2. Adam with **clipnorm=1.0** .
3. EarlyStopping + ReduceLROnPlateau on **val_loss** .
4. After both phases, **export the better checkpoint** of phase1 vs phase2 by **val_loss** — never ship the last crashed epoch.

On the stable run, Phase-2 train-loss spike vs end of Phase-1 was $\approx -1\%$ (**Softmax**) and $\approx -2\%$ (**CORAL**) — i.e. no collapse.

8. Training operations

Hardware profile (this product build)

Resource	Observation
Local GPU	None
Local RAM	~7 GB
Training venue	Local CPU for stable B0 runs; Kaggle GPU recommended for heavier upgrades

Train vs serve hardware



Artifact layout

```
artifacts/experiments/softmax_stable/  
  config.json  
  phase1/best_softmax.keras  history.csv  
  phase2/best_softmax.keras  history.csv  
  best_selection.json  
  softmax_final.keras  
  history.json  
  
artifacts/experiments/coral_stable/  
  ... (same pattern with best_coral.keras)  
  
models/  
  softmax_effb0.keras  
  coral_effb0.keras  
  best_model.keras      ← copy of winner  
  meta.json
```

Stable selection:

- Softmax: **phase2**, best val_loss $\approx$ **0.906**
- CORAL: **phase2**, best val_loss $\approx$ **0.271**

CLI entrypoints

```
uv run python scripts/download_dataset.py
uv run python scripts/run_eda.py           # figures + splits
uv run python scripts/data_sanity.py
uv run python scripts/train_softmax.py     # reuses data/processed/*.csv
uv run python scripts/train_coral.py
uv run python scripts/evaluate_export.py   # test metrics + best_model + meta
uv run streamlit run app/streamlit_app.py
```

Package code lives under `src/dr_detect/` — scripts are thin wrappers.

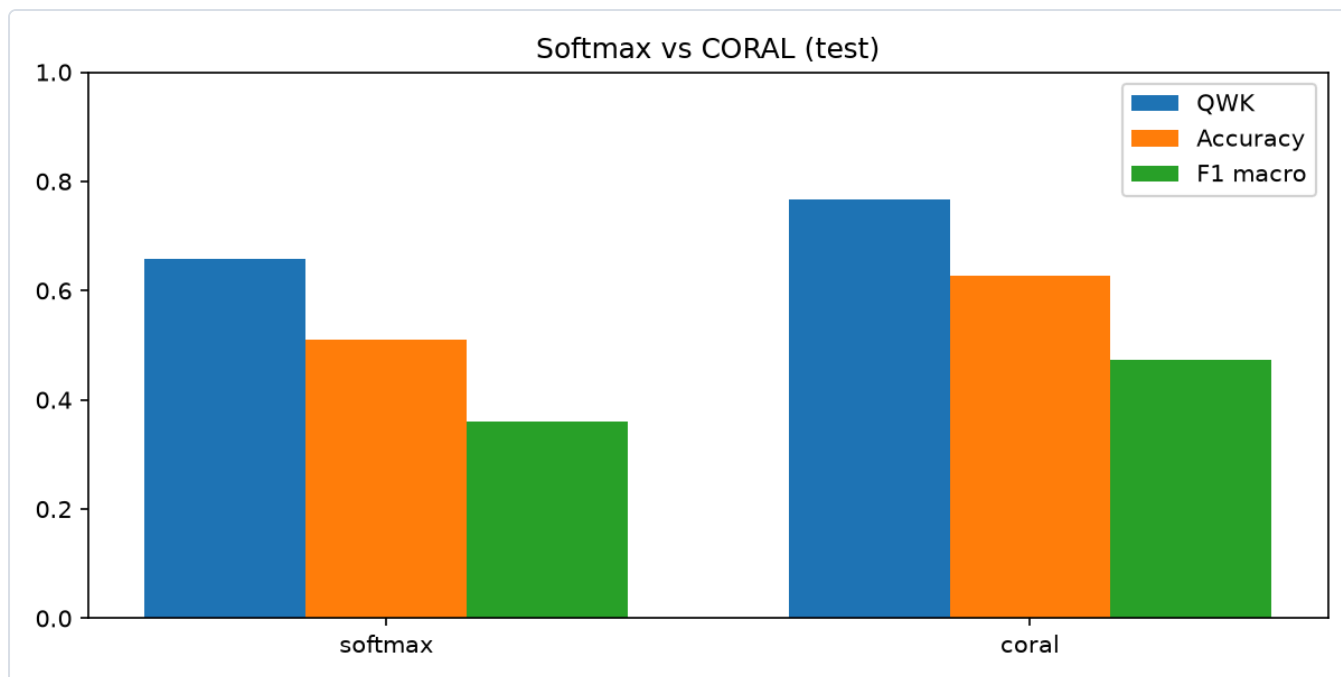
9. Results

Held-out **test set N = 413** (never used for training or early stopping). Metrics from `models/meta.json` after **EXP-ACC-070** (APTOS extra train, label smoothing off, last 30 non-batch-norm layers, up to 12 epochs per phase, checkpoint by validation accuracy, CORAL threshold 0.525 chosen on validation, horizontal-flip test-time average). Trained on a Colab GPU.

Headline comparison

Model	Accuracy	Macro-F1	Weighted-F1	QWK
Softmax	0.625	0.532	0.625	0.724
CORAL (winner)	0.651	0.508	0.636	0.738

The previous deployed CORAL model (EXP-ACC-001) was Acc 0.627 / QWK 0.768. This run raised exact accuracy and lowered kappa. Softmax recovered from 0.511 to 0.625. CORAL is still the deployed head because exact accuracy is higher. 70% exact accuracy was not reached (269 of 413).



Error structure (adjacent vs far)

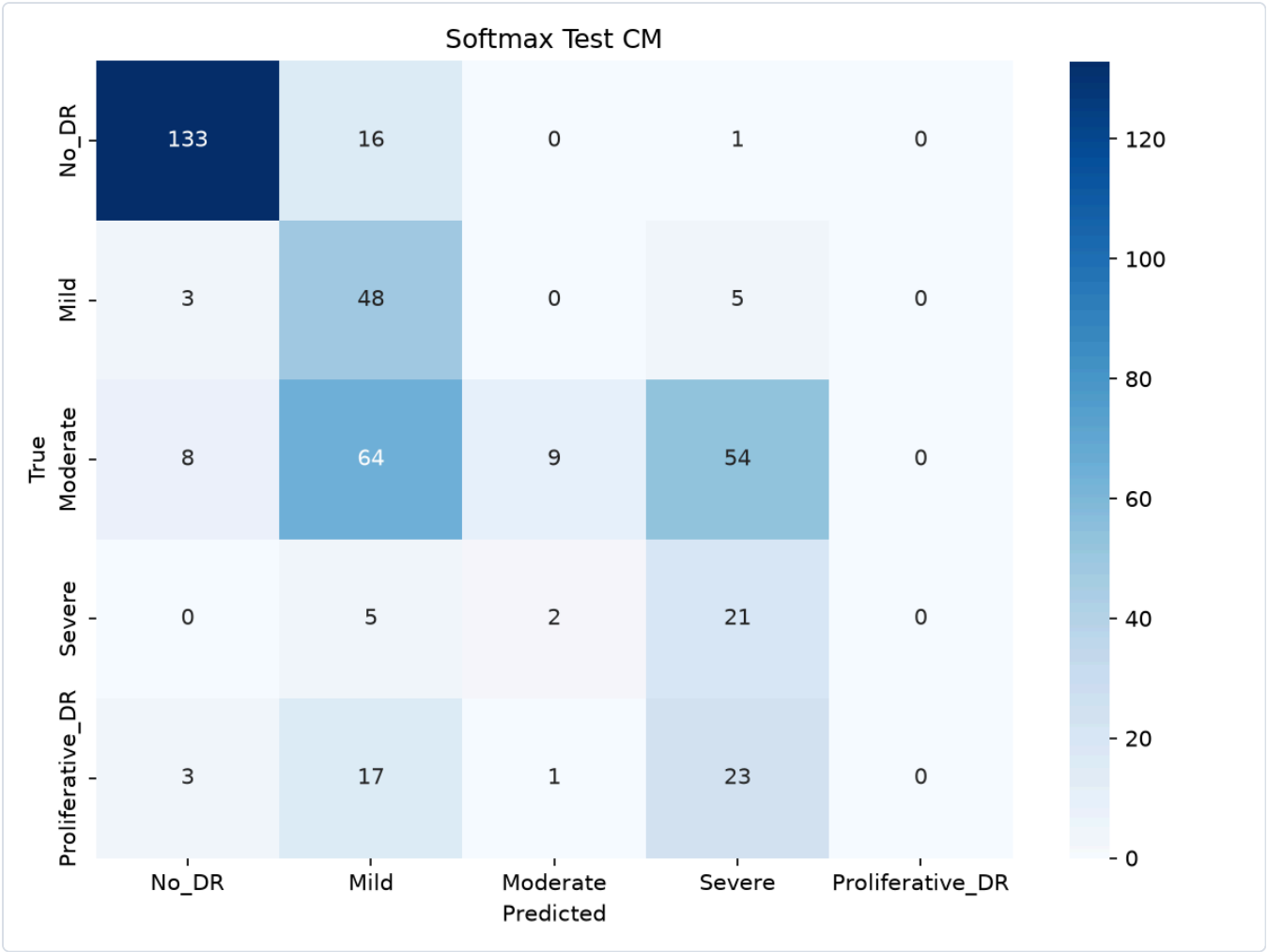
Model	Exact	Adjacent (± 1)	Far (≥ 2)	Mean abs error
Softmax	0.625	0.223	0.153	0.564
CORAL	0.651	0.249	0.099	0.472

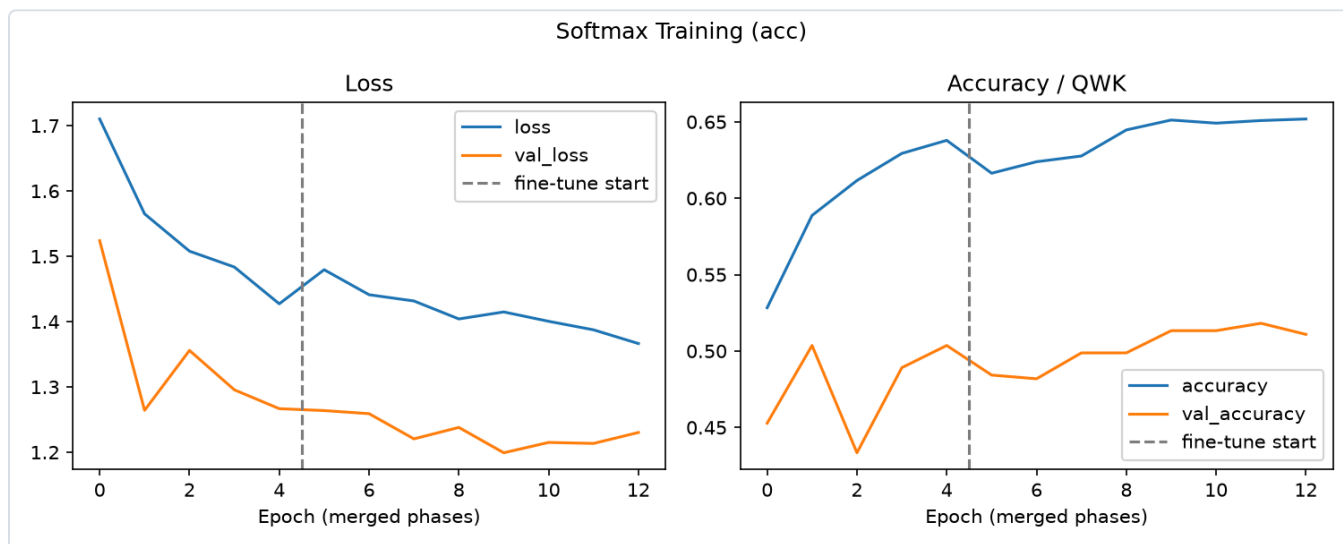
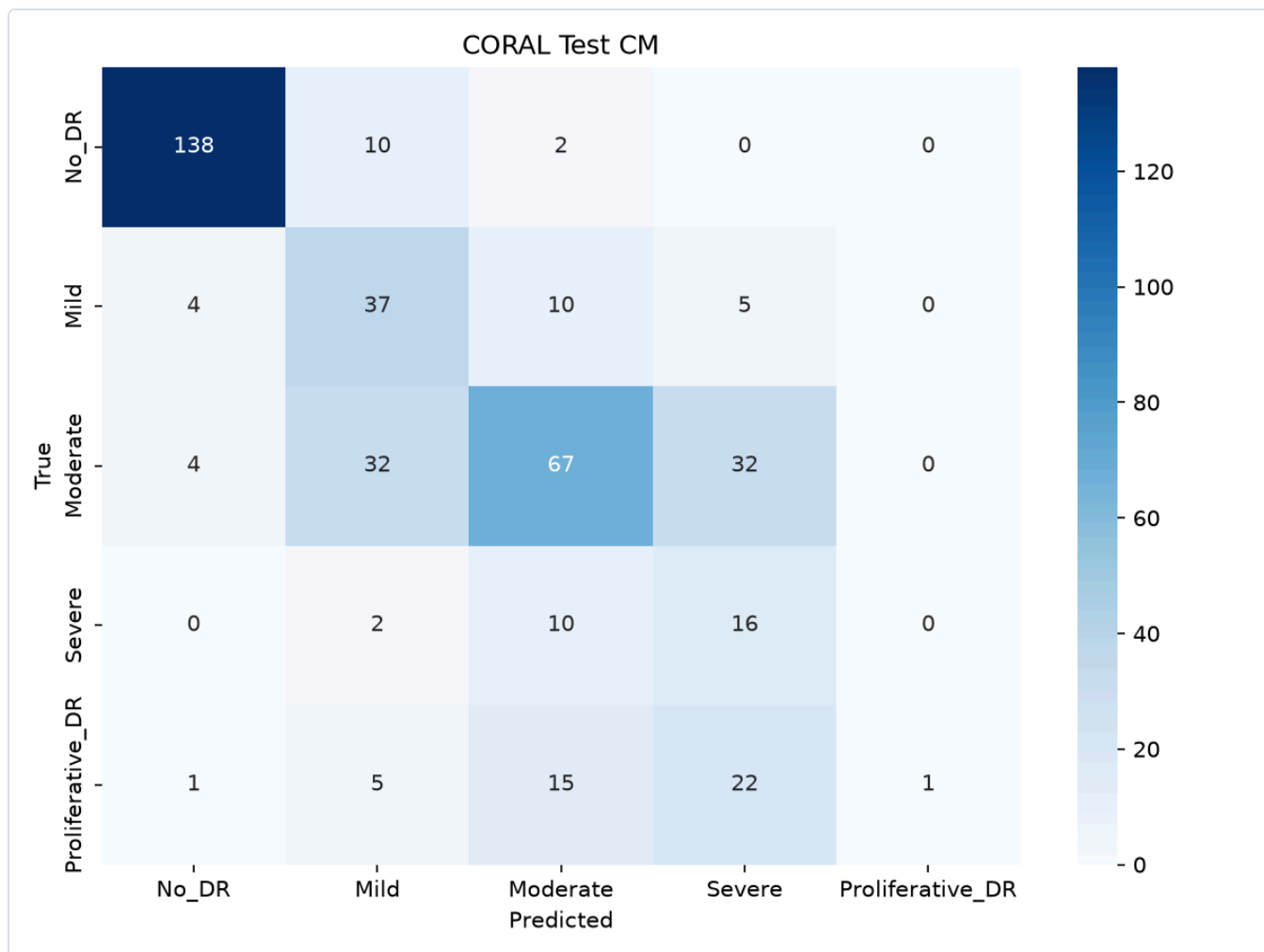
CORAL far-error is **0.099** (it was 0.082 on EXP-ACC-001). Proliferative recall on the test report is **0.07** — still too low for unsupervised screening.

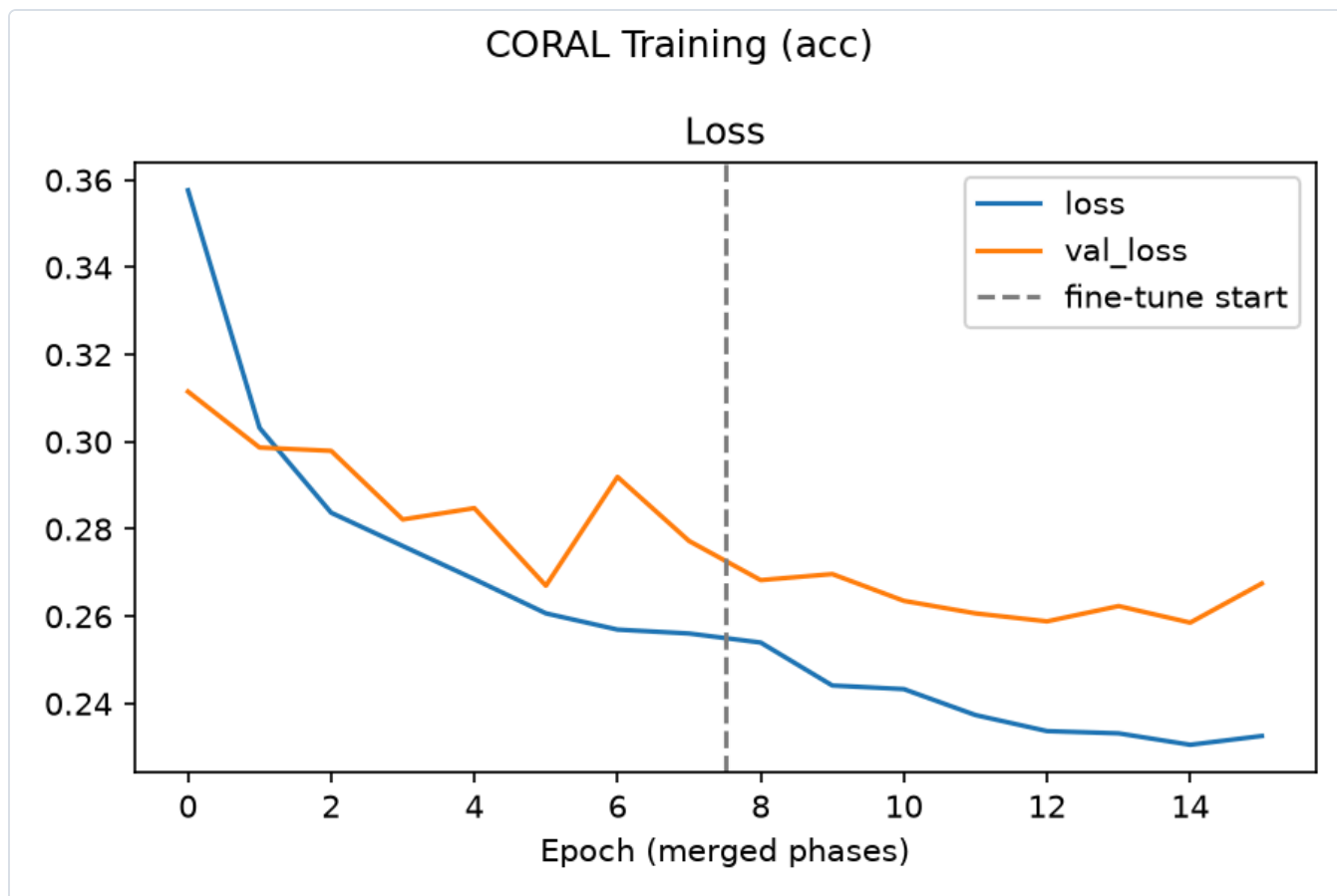
Phase-2 stability

Train-loss change from the last Phase-1 epoch to the first Phase-2 epoch: Softmax about **+3.6%**, CORAL about **-0.8%**. Both stay under the 20% spike limit.

Confusion matrices and curves



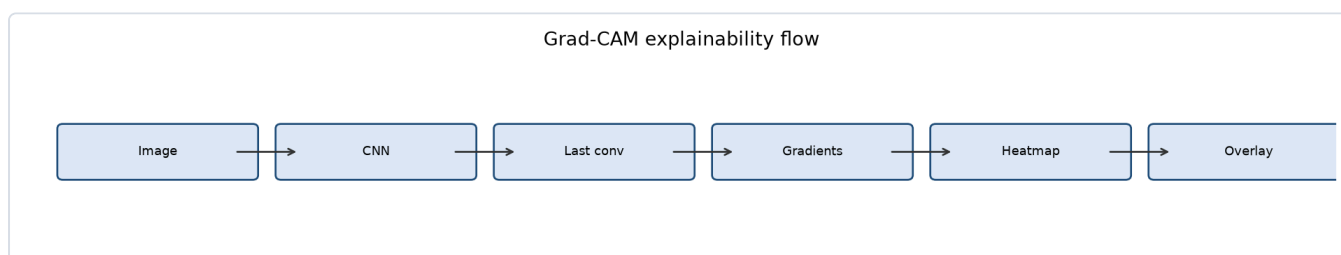


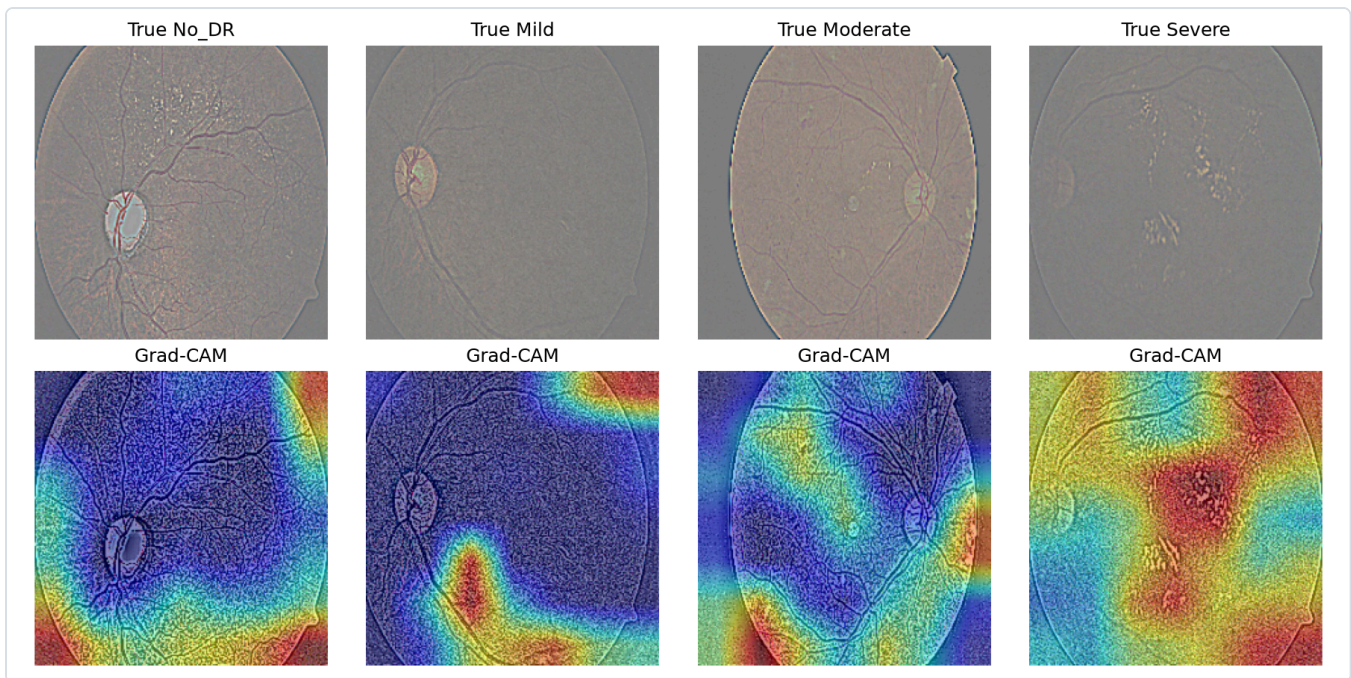


Curve plots mark the Phase-1 → Phase-2 boundary so “epoch 3 collapse” is not misread as a data bug.

Explainability

Grad-CAM on the last convolutional block; overlays should concentrate on vessels / lesions rather than borders.

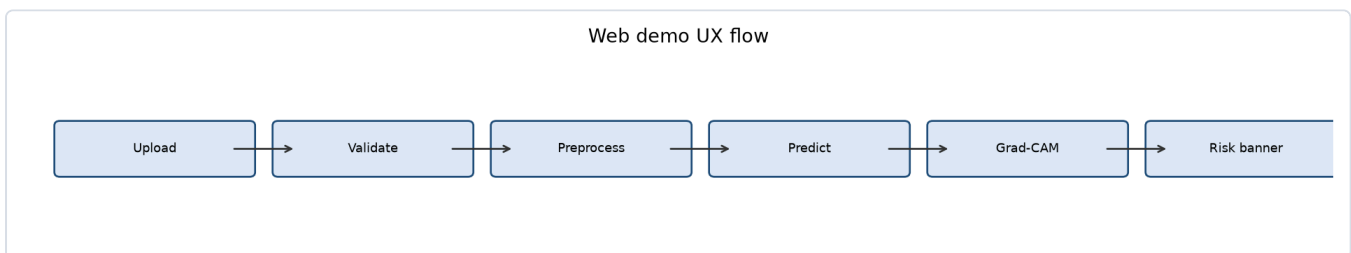




Product interpretation

- Prefer **CORAL** when ordinal triage quality (QWK, far-error) is the primary KPI.
- Softmax remains useful as a **baseline** and has stronger macro-F1 on this split — useful for per-class debugging (Severe / PDR remain hard for both).

10. Demo and product UX



Launch

```
uv run streamlit run app/streamlit_app.py
```

User flow

1. App loads `models/best_model.keras` + `models/meta.json` (cached).
2. User uploads a fundus image (or picks a sample).
3. Pipeline runs the same preprocess as training.
4. UI shows: predicted stage, risk-oriented framing, preprocess previews, Grad-CAM overlay.
5. Persistent **disclaimer**: screening assistant only — not for diagnosis.

Deployment contract

Any future model swap must:

1. Write weights to `models/best_model.keras`.
2. Update `models/meta.json` with `coral` boolean, `class_names`, metrics, preprocess knobs.
3. Keep preprocess identical to training (`CFG` values).

11. Limitations

Limitation	Impact
Colab GPU schedule (up to 12+12, early stop on validation accuracy)	Still short of a 70% exact-match target
Severe / Proliferative scarcity	Weaker recall on rare high-risk grades
No patient-level split IDs	Possible correlated eyes across splits
Camera / clinic domain shift	Metrics may drop on external cameras
Single public dataset	Not a multi-site clinical validation
Grad-CAM is explanatory, not proof	Heatmaps aid trust; they are not ground-truth lesions

Treat published Acc / QWK as **benchmarks on this split**, not clinical performance claims.

12. Accuracy upgrade roadmap

EXP-ACC-001 trained the items marked **done** below. The test CSV stayed held out. Items marked **future** were not trained.

Priority stack

Priority	Change	Status
P0	Richer head: GAP → Dense(512) → Dropout → Dense(256) → Dropout → output	Done (both heads)
P0	Rare-class oversampling of Mild, Severe, and Proliferative on the pooled train split only	Done (each matched the majority count, 1882; fit rows 8831)
P1	Softmax label smoothing 0.1	Tried in EXP-ACC-001, off in EXP-ACC-070
P1	Horizontal-flip average at evaluation	Done
P1	Longer BN-safe schedule (12+12 on Colab GPU)	Done
P2	Select checkpoints by validation accuracy	Done in EXP-ACC-070
P2	Unfreeze last 30 non-BN layers	Done
P2	Deduplicated APTOS 2019 images added to train only	Done (2538 kept, 1124 duplicates dropped)
P3	EfficientNetB3 on a GPU	Future
P3	External test on Messidor-2 or a held-out EyePACS subset	Future
P3	Softmax and CORAL ensemble	Future

What EXP-ACC-001 met

1. Phase-2 train-loss spike stayed under 20% (Softmax +3.6%, CORAL −0.8%).
 2. EXP-ACC-070 CORAL test accuracy **0.651** (was 0.627) and QWK **0.738** (was 0.768). Far-error is 0.099.
 3. Proliferative recall is still low (0.07).
 4. Same `data/processed/*.csv` splits. Test images were not added to training.
-

13. Glossary

Term	Meaning
Fundus	Photo of the retina through the pupil
ICDR	International Clinical Diabetic Retinopathy scale (0–4)
Transfer learning	Reuse ImageNet weights; adapt to fundus
Fine-tune	Unfreeze top backbone layers at low LR
BatchNorm (BN)	Normalization layers with running mean/variance — keep frozen when fine-tuning
Softmax	Multi-class probability head
CORAL	Consistent Rank Logits — ordinal cumulative thresholds
QWK	Quadratic Weighted Kappa — ordinal agreement metric
Grad-CAM	Gradient-weighted Class Activation Mapping — heatmap explainability
CLAHE	Contrast Limited Adaptive Histogram Equalization
Ben Graham prep	Illumination normalization popularized in Kaggle DR
Class weight	Per-class loss multiplier to counter imbalance
Screening assistant	Prioritizes review; does not diagnose

14. How to reproduce

Environment

```
cd diabetic-retinopathy-detection
uv sync
# place Kaggle credentials at ~/.kaggle/kaggle.json (chmod 600)
```

Full pipeline (from scratch)

```
uv run python scripts/download_dataset.py
uv run python scripts/run_eda.py
uv run python scripts/data_sanity.py
uv run python scripts/train_softmax.py
uv run python scripts/train_coral.py
uv run python scripts/evaluate_export.py
uv run streamlit run app/streamlit_app.py
```

Interactive walkthrough

`notebooks/DR_Stage_Detection_Complete_Walkthrough.ipynb` — same concepts cell-by-cell, including the BN-safe stability notes.

Export this guide to PDF

```
uv run python scripts/export_product_guide.py
# writes docs/PRODUCT_GUIDE.html and docs/PRODUCT_GUIDE.pdf
```

Source documents (deeper reference)

Doc	Role
<code>docs/PRODUCT_GUIDE.md</code>	This product bible
<code>docs/RESEARCH.md</code>	Research log and method rationale
<code>docs/ARCHITECTURE.md</code>	Mermaid architecture sources
<code>docs/EXPERIMENT_LOG.md</code>	Chronological training runs
<code>docs/DECISION_LOG.md</code>	Design decisions
<code>Notes/KNOWLEDGE_CACHE.md</code>	Distilled course/product notes

Document version: 2026-10-01 · Metrics from EXP-ACC-070 · Test split held out · 70% exact accuracy not reached · B3 and an external test set still future.